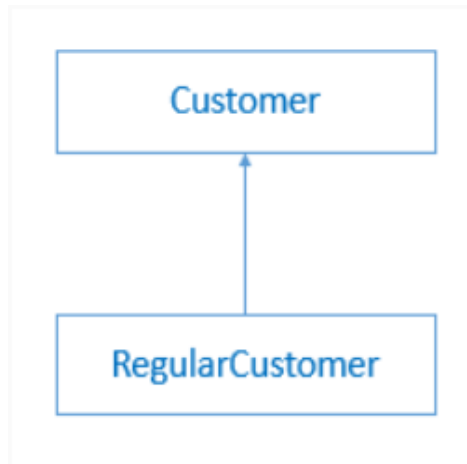


Inheritance

Types of Inheritance

1) Single Inheritance

- In single inheritance, one class is extended by only one class.
- i.e., RegularCustomer extends Customer class.



- Example:

- `public class Customer {`

- `// Parent/Super/Base class`

- `}`

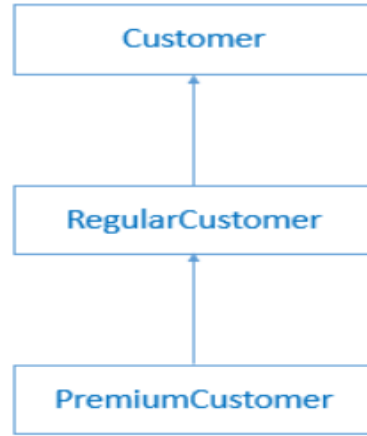
- `public class RegularCustomer extends Customer {` `// RegularCustomer is a Customer`

- `// Child/Sub/Derived class`

- `}`

2) Multilevel Inheritance:

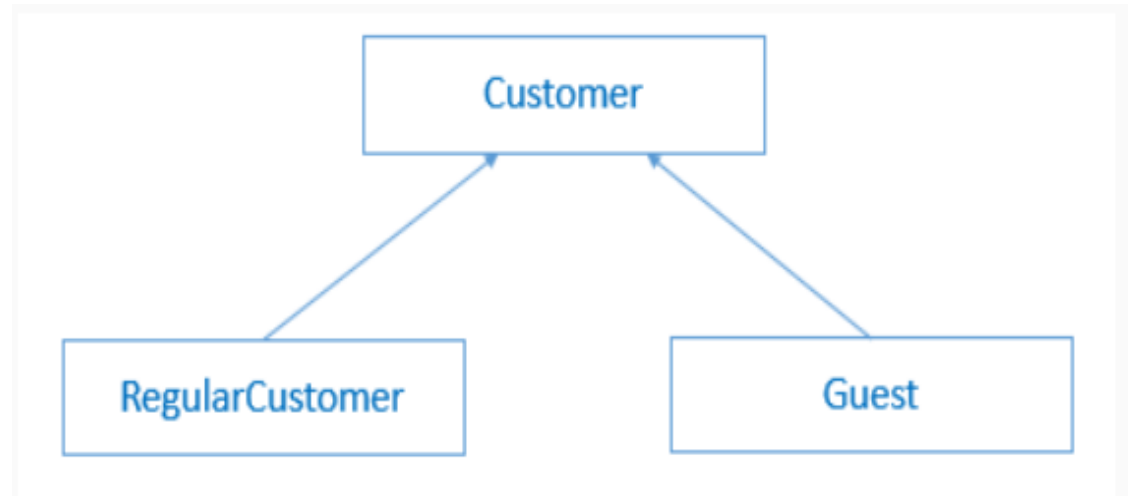
- In multilevel inheritance, a class extends another class which extends another class.
- SwiftFood app has one more category of customers - PremiumCustomers. PremiumCustomers are special type of RegularCustomers who are provided with reward points apart from getting discount.
- In this case, the RegularCustomer class extends the Customer class and the PremiumCustomer class extends the RegularCustomer class. This is an example of Multilevel inheritance



- Example:
- ```
public class Customer {
 //code
}
public class RegularCustomer extends Customer {
 //code
}
public class PremiumCustomer extends RegularCustomer {
 //code
}
```

### 3) Hierarchical Inheritance:

- When more than one class extends the same base class, then that type of inheritance is said to be Hierarchical Inheritance.
- Here, the Customer class is the parent class and the RegularCustomer and Guest classes extend the Customer class. This is an example of Hierarchical Inheritance.



- **Example:**

- `public class Customer {`

- `// code here`

- `}`

- `public class RegularCustomer extends Customer {`

- `// code here`

- `}`

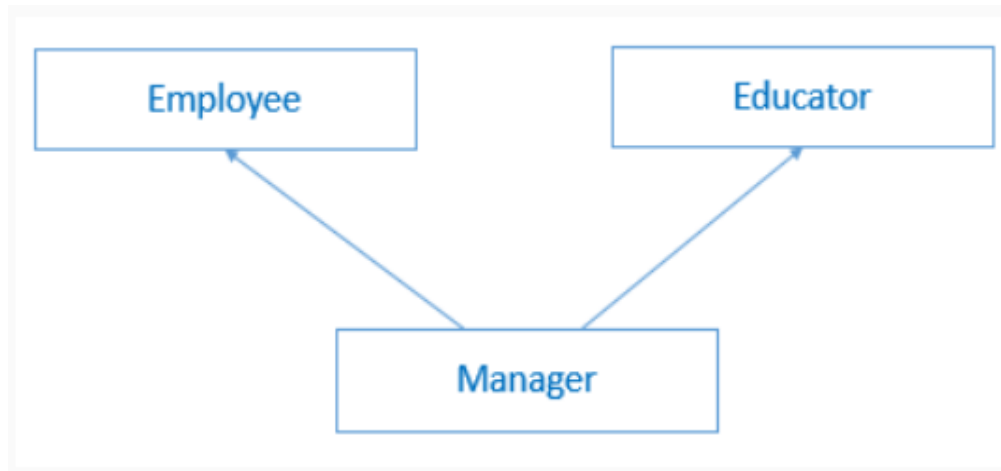
- `public class Guest extends Customer {`

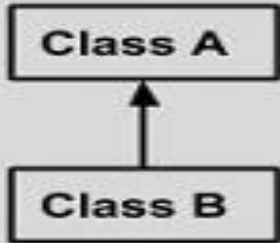
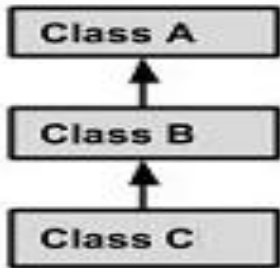
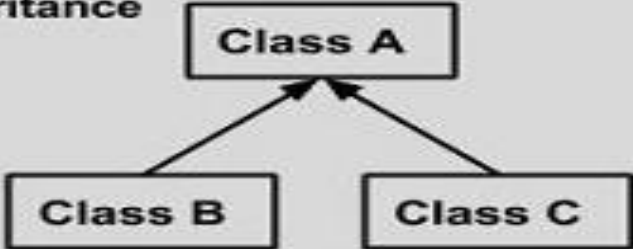
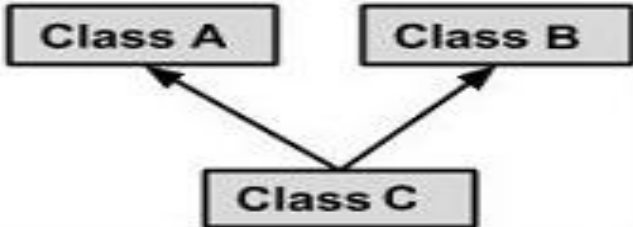
- `// code here`

- `}`

#### 4) Multiple Inheritance:

- In Multiple Inheritance, one class extends multiple classes.
- In the below example, Manager is an Employee as well as an Educator. So, Manager class extends both Employee and Educator class.
- Multiple Inheritance is NOT supported in many object oriented programming languages including Java.



|                          |                                                                                                                                                                     |                                                                                                                                                      |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Single Inheritance       |  <pre> graph BT     B[Class B] --&gt; A[Class A] </pre>                           | <pre> public class A {     ..... } public class B extends A {     ..... } </pre>                                                                     |
| Multi Level Inheritance  |  <pre> graph BT     C[Class C] --&gt; B[Class B]     B --&gt; A[Class A] </pre>   | <pre> public class A { ..... } public class B extends A { ..... } public class C extends B { ..... } </pre>                                          |
| Hierarchical Inheritance |  <pre> graph BT     B[Class B] --&gt; A[Class A]     C[Class C] --&gt; A </pre>   | <pre> public class A { ..... } public class B extends A { ..... } public class C extends A { ..... } </pre>                                          |
| Multiple Inheritance     |  <pre> graph BT     C[Class C] --&gt; A[Class A]     C --&gt; B[Class B] </pre> | <pre> public class A { ..... } public class B { ..... } public class C extends A,B {     ..... } // Java does not support mutiple Inheritance </pre> |

# Advantages of Inheritance

- The main advantages of inheritance are:
  - Code reusability - The codes are defined only once and can be used multiple times. We define the functionalities in the parent class and any number of child classes can use the functionalities at any time.
  - Saves time and effort due to reusability of code.
  - Inheritance makes the application code more flexible to changes.
  - Inheritance results in better organization of codes into smaller and simpler compilation units which makes the code more readable and easy to debug.